

12. Validace rodného čísla

Tato kapitola patří mezi náročnější, neboť předpokládá znalost práce s textovým řetězcem. Naprogramujeme si metodu, která prozkoumá správnost zápisu rodného čísla. Nebo lépe řečeno budeme programovat více metod spolu souvisejících. Pokud by Vám přišla kapitola obtížná, lze ji přeskočit a později se k ní vrátit. Nebylo by však moudré nechat tento vstup rodného čísla zcela bez kontroly. V dřívější kapitole rodné číslo používáme jako klíčový prvek pro rozlišení každého z žáků. Správnost jeho zápisu je tedy více než důležitá.

Formát rodného čísla

Rodné číslo je jednoznačným identifikátorem každého občana České republiky. Používá se ale například i na Slovensku.

Celá problematika kontroly správnosti rodného čísla je rozsáhlá. Proto celý příklad zjednodušíme na nejpoužívanější pravidla. Informace o výjimkách si pak může čtenář v případě zájmu vyhledat a metodu doprogramovat.

Pro kontrolu rodného čísla jsem vybral následující předpoklady:

- Rodné číslo je deseticiferné číslo oddělené lomítkem, má tedy 11 znaků
- Má formát: rmmdd/xxxx
 - rr – rok narození (1989 -> 89)
 - mm – měsíc narození (leden -> 01)
 - pokud se jedná o ženské pohlaví, přičítá se k měsíci číslo 50 (leden -> 51)
 - dd – den narození (3 -> 03)
- xxxx – kontrolní číslice, doplněk k rodnému číslu
 - platí, že číslo rmmddxxxx je celočíselně dělitelné číslem 11.
 -

Vidíme tedy, že **konstrukce rodného čísla má souvislost s datem narození a pohlavím.**

Tvorba statického interface

Validaci rodného čísla jsem se rozhodl tvořit **statickou cestou**. Od jiných našich tříd se bude lišit tím, že z ní **nelze tvořit instance** pomocí konstruktoru. **Nelze z ní tvořit objekty**. Tato třída tedy během programu **existuje pouze v jediném exempláři**. A to stačí, protože neustále vykonává tytéž operace a ani do sebe neukládá nějaká pro uživatele jedinečná data.

Statické třídy se v dnešní době moc nepoužívají. Není k tomu v podstatě důvod, neboť každá statická třída se dá naprogramovat i nestaticky. Přesto se s nimi můžete setkat, a tak nám ukázka použití jistě bude prospěšná.

Obecně statické třídy používám, když potřebuji implementovat funkcionalitu, která je během programu stále neměnná. Vezměme si například třídu kalkulačka. Vždy bude vykonávat tytéž funkce. Sčítání a odčítání funguje stejně na zelené dětské kalkulačce i na velkém vědeckém přístroji. Stačí nám pro všechny uživatele jedna jediná, netřeba vytvářet instance. Jinak přemýšlíme v momentě, kdy do kalkulačky požadujeme přidat paměť na několik čísel. V tom okamžiku samozřejmě každý uživatel chce ukládat vlastní čísla. Zde by tedy statickou třídu nebylo možné použít.

Vraťme se nyní nazpět k našemu problému validace rodného čísla. Kde kód umístit?

Tuto funkci bych mohl „připnout“ například do třídy Zak.java. Pokud bych ale potřeboval použít validaci i jinde v programu (například při doprogramování evidence pedagogických zaměstnanců), celý kód bych musel kopírovat, a to se prostě nedělá.

Vytvořím tedy třídu **Validace**, která bude řešit nejen validaci rodného čísla, ale obecně jakékoli validace, pokud bude potřeba. Statická třída se v kódu Javy speciálně neoznačuje. Označují se pouze metody, které do ní programujeme. Konstruktor samozřejmě vynecháváme, ve statické třídě nedává jeho přítomnost smysl.

Statická třída Validace bude viditelná a použitelná ve všech ostatních class. Na druhou stranu to ale není úplně potřeba. Stačí přece, aby byla viditelná z těch tříd, kde je potřeba validovat rodná čísla. Proto půjdeme ještě dál.. nevytvoříme klasickou statickou třídu, ale tzv. **interface**. Možná cítíte, že tento termín má něco společného s uživatelským rozhraním. A taky že má. Nicméně my jej nyní použijeme trochu jinak. Nebudu tady rozvádět pojem **vícenásobná dědičnost**, abych čtenáře zbytečně neodradil. Co kdybychom si ale řekli, že třídu označenou jako interface můžeme implementovat pouze do konkrétních námi zvolených tříd. Do tříd, které požadují validaci rodného čísla. Ano, toto nám interface umožní. Podle domluvené konvence tedy místo class Validace vytvoříme **interface IValidace**.

Metoda: Rodné číslo - kontrola

Základní statickou metodu našeho nového Interface pojmenujeme **rodneCisloKontrola**. Bude vracet buď true, nebo false. Podle toho, jestli je rodné číslo ve správném tvaru či nikoli. Protože je problematika validace složitá, pro přehlednost rozdělíme funkcionalitu do více různých metod. Každá z nich bude rovněž statická a bude vracet true nebo false. Základní metoda rodneCisloKontrola si pak dílčí metody zavolá a u každé ověří, zda podmínka je či není splněna. Samozřejmě platí, že pokud platí všechny výše uvedené podmínky, rodné číslo je validní.

Rovněž stojí za pozornost, že dílčí metody jsou označeny jako **private**, nelze je tedy volat odkudkoli. Jsou zde pouze pro interní potřeby interface IValidace.java. Jedinou **public** metodou je metoda rodneCisloKontrola.

Základní statická metoda rodneCisloKontrola (interface IValidace.java)

```
public interface IValidace {  
  
    public static boolean rodneCisloKontrola(String rc, boolean muz, LocalDate dn)  
    {  
        if (!overFormatRc(rc)) {  
            System.out.println("Error: chyba ve formátu rodného čísla.");  
            return false;  
        }  
        if (!overDelitelnost11(rc)) {  
            System.out.println("Error: chyba v dělitelnosti rodného čísla 11ti.");  
            return false;  
        }  
        if (!overMesic(rc, muz, dn)) {  
            System.out.println("Error: nesedí měsíc z rodného čísla s měsícem v datu narození.");  
            return false;  
        }  
        if (!overDen(rc, dn)) {  
            System.out.println("Error: nesedí den z rodného čísla se dnem v datu narození.");  
            return false;  
        }  
        if (!overRok(rc, dn)) {  
            System.out.println("Error: nesedí rok z rodného čísla s rokem v datu narození.");  
            return false;  
        }  
        return true;  
    }  
}
```

Poznámka 1.: Metoda je pro studijní účely doplněna o výpisy konkrétních chybových stavů. V ostrém nasazení bychom výpisy vymazali a ponechali pouze návratovou hodnotu true – false.

Poznámka 2: Řádek s výpisem erroru je v kódu vždy výše než return hodnota. Pomocí return vyskakujeme z metody okamžitě pryč. Proto kdyby byly tyto řádky obráceně, na výpis by vůbec nedošlo.

Poznámka 3: Kód je koncipován tak, že i kdyby validování vykazalo více chyb, vždy se dozvíme pouze o té první v pořadí.

Jak je patrné, do metody vstupuje nejen rodné číslo, ale rovněž pohlaví a rok narození. Tyto tři parametry jsou pro validaci nezbytné. Neboť pracujeme s datem, nezapomeneme nahoře nainportovat naši již známou třídu LocalDate.

Nyní se zaměříme na dílčí statické metody. Každá bude řešit jinou podmínku pro rodné číslo. Při splnění všech podmínek bude rodné číslo vyhodnoceno jako true, tedy v pořádku. Popis fungování metody bude vždy součástí komentářů daného kódu.

Metoda: Ověř formát rodného čísla

Metoda **overFormatRc** ověřuje, zda má rodné číslo 11 znaků, a to 6 cifer, lomítko, 4 cifry.

Metoda overFormatRc (interface IValidace.java)

```
private static boolean overFormatRc(String rc){  
    //ověření, zda má rodné číslo 11 znaků  
    if (rc.length()!=11)  
        return false;  
  
    //ověření, zda je na šestém znaku znak /  
    if (rc.charAt(6) != '/')  
        return false;  
  
    //ověření, zda na ostatních znacích je číslice  
    for (int i=0; i < rc.length();i++)  
    {  
        char aktualniZnak = rc.charAt(i);  
        if (i != 6) {  
            if (!Character.isDigit(aktualniZnak))  
                return false;  
        }  
    }  
  
    //pokud je formát rodného čísla v pořádku, metoda vrátí true  
    return true;  
}
```

Metoda: Ověř dělitelnost 11ti

Metoda **overDelitelnost11** ověřuje, zda je rodné číslo (po odstranění lomítka) beze zbytku dělitelné číslem 11.

Metoda overDelitelnost11 (interface IValidace.java)

```
private static boolean overDelitelnost11(String rc)
{
    //Odstraníme z rodného čísla lomítko a následně jej převedeme na číslo
    String rcBezLomitka = rc.replace( target: "/", replacement: "");
    long rcCislo = Long.parseLong(rcBezLomitka);

    if (rcCislo%11==0)
        return true;
    else
        return false;
}
```

Metoda: Ověř měsíc narození z rodného čísla

Metoda **overMesic** ověřuje shodu měsíce z data narození s měsícem uvedeném v rodném čísle. Pokud se jedná o ženské pohlaví, přičítá se k měsíci v rodném čísle 50 (např. březen není 3, ale 53)

Metoda overMesic (interface IValidace.java)

```
private static boolean overMesic(String rc, boolean muz, LocalDate dn)
{
    int rcMesic = Integer.parseInt(rc.substring(2,4)); //měsíc získaný z rodného čísla
    int dnMesic = dn.getMonth().getValue(); //měsíc získaný z data narození

    //pokud se jedná o ženu, přičteme k měsíci číslo 50
    if (!muz)
        dnMesic+=50;

    if (rcMesic == dnMesic)
        return true;
    else
        return false;
}
```

Metoda: Ověř den narození z rodného čísla

Metoda **overDen** ověřuje shodu dne z data narození se dnem uvedeným v rodném čísle.

Metoda overDen (interface IValidace.java)

```
private static boolean overDen(String rc, LocalDate dn)
{
    int rcDen = Integer.parseInt(rc.substring(4,6)); //den získaný z rodného čísla
    int dnDen = dn.getDayOfMonth(); //den získaný z data narození

    if (rcDen == dnDen)
        return true;
    else
        return false;
}
```

Metoda: Ověř rok narození z rodného čísla

Metoda **overRok** ověřuje shodu roku z data narození s rokem uvedeným v rodném čísle.

Mějme na paměti, že v datu narození je rok uložen v plném formátu (např. 1998), zatímco v rodném čísle ve zkráceném formátu (98). Jedno z čísel je tedy nutné před srovnávání upravit.

Metoda overRok (interface IValidace.java)

```
private static boolean overRok(String rc, LocalDate dn)
{
    int rcRok = Integer.parseInt(rc.substring(0,2)); //rok získaný z rodného čísla

    //rok získaný z data narození
    String dnRokString = String.valueOf(dn.getYear());
    int dnRok = Integer.parseInt(dnRokString.substring(2,4));

    if (rcRok == dnRok)
        return true;
    else
        return false;
}
```

Implementace a otestování metody rodneCisloKontrola

Vyzkoušíme, zda metoda rodneCisloKontrola funguje správně. Zavoláme ji napřímo z Main.java. Takto sice v reálu metodu volat nebudeme, nicméně pro účely testování je vše dovoleno.

V první fázi implementujeme interface IValidace. Tu provedeme v class Třída, neboť ověřování rodného čísla využijeme právě zde – při tvoření a zařazování nového žáka do třídy.

Implementace Ivalidace (Trida.java)

```
public class Trida implements IValidace {
    private ArrayList<Zak> zaci;
    private byte rocnik;
    private String tridni;
```

Následně upravíme metodu pro tvorbu žáka. Návrátová hodnota metody nás bude informovat, zda přidání žáka proběhlo úspěšně či nikoli.

Metoda pridejNovehoZaka - úprava (Trida.java)

```
public String pridejNovehoZaka(String jmeno, String prijmeni, String rodne_cislo, LocalDate datum_narozeni, boolean chlapec)
{
    if (IValidace.rodneCisloKontrola(rodne_cislo, chlapec, datum_narozeni)) {
        if (!jeZakVeTride(rodne_cislo)) {
            zaci.add(new Zak(jmeno, prijmeni, rodne_cislo, datum_narozeni, chlapec, rocnik));
            if (chlapec)
                return String.format("Žák %s %s byl přidán do %d. třídy", jmeno, prijmeni, rocnik);
            else
                return String.format("Žákyně %s %s byla přidána do %d. třídy", jmeno, prijmeni, rocnik);
        }
        else
            return String.format("Chyba přidání nového žáka: Bylo zjištěno, že žák/žákyně s rodným číslem %s " +
                "již ve třídě existuje.", rodne_cislo);
    }
    else
        return String.format("Chyba: byla zjištěna chyba v datu narození nebo rodném čísle");
}
```

Volání a ověřování metody pro přidávání nového žáka

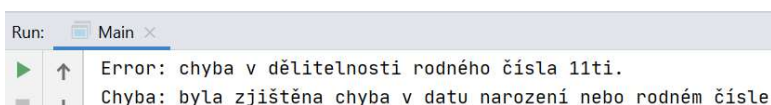
Metodu budeme volat, tak jako dříve, z class Main.java. Pamatujme, že v případě zjištění chyby rovného čísla se nám zobrazí vždy pouze první testovaná chyba. Zobrazení více chyb by stejně nemělo valného smyslu, jelikož v tomto případě jeden překlep může klidně znamenat vyvolání 3 různých chyb.

Testování metody pridejNovehoZaka - s chybou (Main.java)

```
public class Main {
    public static void main(String[] args) {

        Trida t1=new Trida((byte)2,"Gabriela Ohmová");

        System.out.println(t1.pridejNovehoZaka( jmeno: "Vlasta", prijmeni: "Výborná", rodne_cislo: "110524/5652",
            LocalDate.of( year: 2011, month: 5, dayOfMonth: 24), chlapec: false));
```

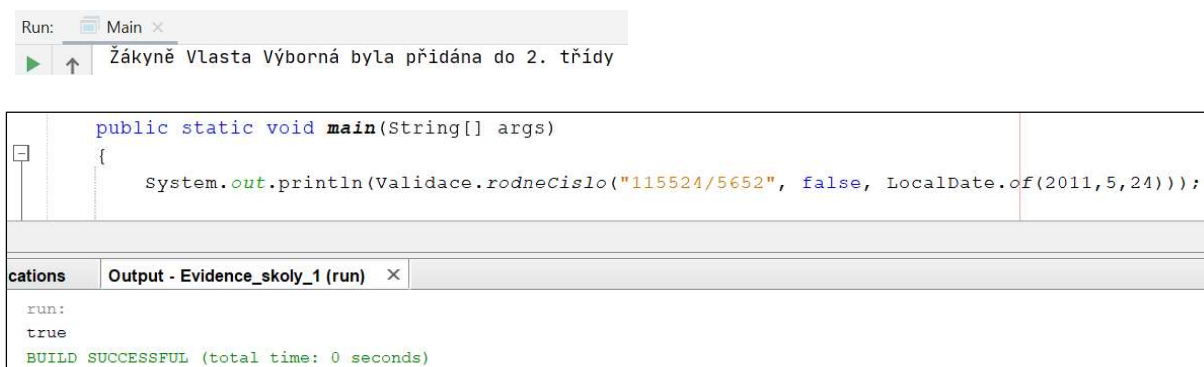


```
Run: Main ×
Error: chyba v dělitelnosti rodného čísla 11ti.
Chyba: byla zjištěna chyba v datu narození nebo rodném čísle
```

Poznámka: V našem příkladu volání je očividně problém s měsícem, kdy dívka narozená v květnu musí mít v rodném čísle na třetí a čtvrté pozici čísla 55, nikoli 05. To první co se zachytilo je však dle výpisu chyba při dělení 11ti. Přitom když opravíme v rodném čísle měsíc, upravíme tím zároveň použitý dělenec a vše bude vycházet.

Testování metody `pridejNovehoZaka` - bez chyby (`Main.java`)

```
System.out.println(t1.pridejNovehoZaka( jmeno: "Vlasta", prijmeni: "Výborná", rodne_cislo: "115524/5652",  
    LocalDate.of( year: 2011, month: 5, dayOfMonth: 24), chlapec: false));
```



```
public static void main(String[] args)
{
    System.out.println(Validace.rodneCislo("115524/5652", false, LocalDate.of(2011,5,24)));
}

cations  Output - Evidence_skoly_1 (run)  X
run:
true
BUILD SUCCESSFUL (total time: 0 seconds)
```

Nyní je vše v pořádku. Metoda `rodneCisloKontrola` z interaface `IValidace` funguje. Navíc, kdykoli v budoucnu program obohatíme o další třídy, ve kterých bude potřeba ověřovat rodná čísla, stačí již hotový interface jednoduše znovu použít.

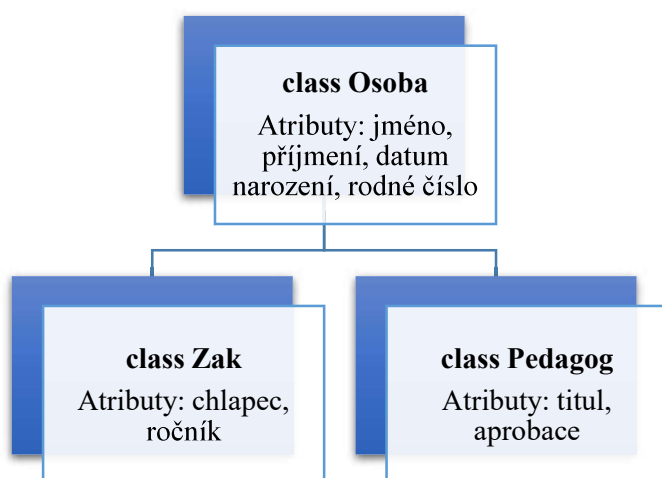
V následujících kapitolách do naší školy konečně přivítáme učitelský sbor. Využijeme k tomu dědičnosti..

13. Dědičnost

Je čas přidat do našeho projektu třídu Pedagog. Mezi atributy by mohly patřit jméno, příjmení, titul, rodné číslo, datum narození a seznam aprobací.

Z pohledu našeho programu vidíme styčné body mezi objekty typu Zak a Pedagog. Jedná se o atributy jméno, příjmení, rodné číslo a datum narození. Abychom předešli duplikování kódu v programu, můžeme použít dědičnost. Níže uvádím jedno z možných řešení.

Dědická struktura použitá v našem programu



V tomto řešení je třída Osoba brána jako předek. Třídy Zak a Pedagog mají roli potomků. V rámci dědičnosti tedy platí, že potomek vždy dědí veškeré atributy předka, zároveň může být o něco rozšířen.

Můžete namítnout, že objekt třídy Osoba nebude v našem příkladu nikdy vytvořen. Má smysl tvořit pouze žáky či pedagogy. Je to pravda. Třída Osoba je pro nás vlastně jakousi pomocnou konstrukcí. Proto, pokud neplánujeme z naší třídy vytvářet instance, lze ji označit klíčovým slovem **abstract**. Nikde není napsáno, že hierarchie dědičnosti musí obsahovat abstraktní třídu. Nemusí. V našem případě je to ale vhodné, proto abstraktní třídu použijeme.

Dědický vztah Osoba – Žák

Dědičnost jsme vzhledem k posloupnosti budování učiva doposud v našem programu nepoužili. Čeká nás nyní tedy nepříjemný úkon – rozbit již existující definici třídy Zak na dvě třídy. V praxi bychom si výhody použití dědičnosti jistě uvědomili již během návrhu programu a tomuto procesu se tak vyhnuli:

- Vytvoříme novou třídu Osoba a v ní privátní atributy dle návrhu výše.
- Třídu Osoba označíme klíčovým slovem **abstract**.
- V definici třídy Zak pomocí klíčového slova **extends** určíme předka – třídu Osoba.

- Ze třídy Zak odstraníme atributy, které dle návrhu výše do ní nepatří, tyto přesuneme do třídy Osoba.
- Metody get a set vztahované k atributům ve třídě Osoba rovněž přesuneme ze třídy Zak do třídy Osoba

Atributy abstraktní třídy Osoba (Osoba.java)

```
package BalicekProjektu;
import java.time.LocalDate;

public abstract class Osoba {
    private String jmeno;
    private String prijmeni;
    private String rodneCislo;
    private LocalDate datumNarozeni;

    public String getJmeno() { return jmeno; }

    public String getPrijemni() { return prijmeni; }
    public void setPrijemni(String prijmeni) { this.prijmeni=prijmeni; }

    public String getRodneCislo() { return rodneCislo; }
}
```

Úprava třídy Zak - nově poděděné ze třídy Osoba

```
public class Zak extends Osoba{

    private boolean chlapec;
    private byte rocnik;

    public byte getRocnik() { return rocnik; }
    public void setRocnik(byte rocnik) { this.rocnik=rocnik; }

    public boolean isChlapec() { return chlapec; }
    public void setChlapec(boolean chlapec) { this.chlapec = chlapec; }
```

Konstruktory v rámci dědičnosti

Naprogramovat konstruktor třídy Osoba není problém. Osoba má 4 proměnné, u konstruktoru tedy použijeme 4 parametry.

Konstruktor (Osoba.java)

```
public Osoba(String jmeno, String prijmeni, String rodneCislo, LocalDate datumNarozeni) {
    this.jmeno = jmeno;
    this.prijmeni = prijmeni;
    this.rodneCislo = rodneCislo;
    this.datumNarozeni = datumNarozeni;
}
```

Zajímavější na celé věci je, že tento konstruktor nelze zavolat – **není možné vytvořit instanci třídy Osoba**, a to přesto, že konstruktor je public. Výše jsme totiž třídu označili jako abstraktní. Právě proto, aby nebylo možno z ní tvořit instance. Konstruktor je zde pouze **pro potřeby potomků dané třídy**, kteří budou moci tuto metodu použít díky zákonitostem dědičnosti.

Pojďme se podívat, jak se prakticky změní konstruktor (y) u poděděné třídy Zak.

Úprava konstruktoru Zak (Zak.java)

```
//Konstruktor
public Zak(String jmeno, String prijmeni, String rodneCislo, LocalDate datumNarozeni, boolean chlapec, byte rocnik) {
    super(jmeno, prijmeni, rodneCislo, datumNarozeni);
    this.chlapec = chlapec;
    this.rocnik = rocnik;
}

//Přetížený konstruktor
public Zak(String jmeno, String prijmeni, String rodneCislo, LocalDate datumNarozeni, boolean chlapec) {
    this(jmeno, prijmeni, rodneCislo, datumNarozeni, chlapec, (byte)1);
}

//Bezparametrický konstruktor
public Zak() {
}
```

Jak je z obrázku patrné, v hlavičce konstruktorů uvádíme veškeré pro nás důležité proměnné. Změna je až v těle konstruktoru – na začátku vidíme řádek označený klíčovým slovem **super**. Pomocí něj v podstatě voláme konstruktor předka, tedy konstruktor třídy Osoba. Konečně jej využíváme, jak jsme si dříve slíbili. Při tvorbě objektu typu Zak si odskočíme do třídy Osoba, která nám s konstrukcí částečně pomůže. Atributy nacházející se striktně v objektu Zak si třída musí vlastním konstruktorem obstarat sama.

U druhého (přetíženého) konstruktoru část super nepíšeme, neboť tento konstruktor v podstatě pouze volá konstruktor první.

Z obrázku výše je dále patrná chyba v prázdném konstrukturu třídy Zak. Problém je, že předek (třída Osoba) prázdný konstruktor neobsahuje. Nepřipouští tedy možnost vytvořit objekt bez plnění proměnných. Řešení je jednoduché. Do třídy Osoba doprogramujeme přetížený prázdný konstruktor.

Bezparametrický konstruktor (Osoba.java)

```
public Osoba() {
}
```

Konstruktory máme vyřešené, nicméně v programu se objevilo více chyb, které zde dříve nebyly. Konkrétně třída Zak přímo nevidí atributy ze třídy Osoba. Nemůže s nimi pracovat ve svých metodách. Není se čemu divit. Proměnné ve třídě Osoba mají modifikátory přístupu nastaven jako private. Jsou zapouzdřeny v objektu a nelze s nimi aktuálně pracovat jinak, než pomocí metod get a set. Situaci bychom mohli vyřešit

přepisem modifikátoru z `private` na `public`. Jak jsme však dříve zmínili, tato změna může přinést další komplikace.

Modifikátor přístupu `protected`

Existuje ještě jedna možnost – použití modifikátoru přístupu **`protected`**. Ten zpřístupní atribut třídám v rámci dědičnosti.

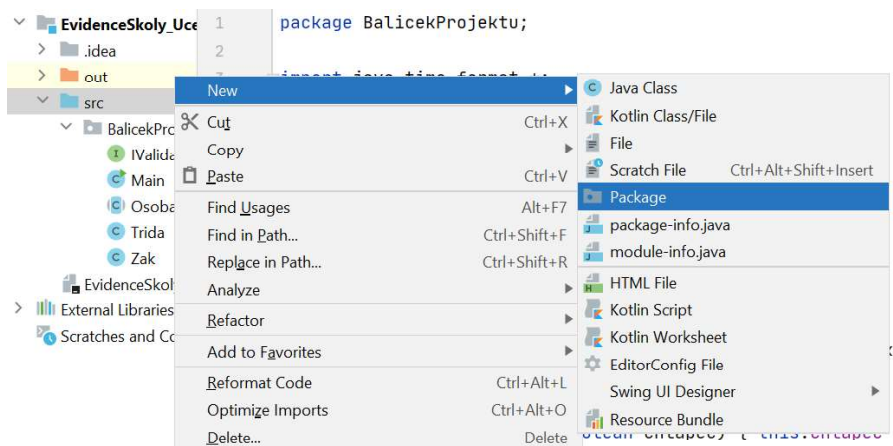
Atributy tříd `Osoba` a `Zak` - úprava modifikátorů přístupu (`Osoba.java`, `Zak.java`)

```
public abstract class Osoba {
    protected String jmeno;
    protected String prijmeni;
    protected String rodneCislo;
    protected LocalDate datumNarozeni;

    public class Zak extends Osoba {
        private boolean chlapec;
        private byte rocnik;
```

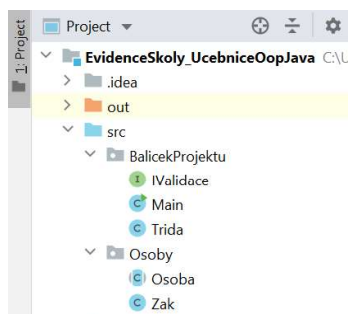
To ale samo o sobě nestačí. Abychom zamezili situaci, že na proměnné přímo uvidí například třída `Program.java`, je nezbytné navíc umístit dědičné třídy do vlastního **balíčku**.

Tvorba nového package v prostředí IntelliJ



V rámci našeho projektu vytvoříme balíček a nazveme jej `Osoby`. Následně do něj přemístíme (přetáhneme myší) jak třídu `Osoba`, tak třídu `Zak`. Tímto je situace vyřešena. Děděné třídy vidí na své atributy, pro třídy mimo tento balíček jsou však proměnné neviditelné. Jsou přístupné pouze přes naše metody `get` a `set`. Práce s nimi je tedy plně v našich rukou. Co povolíme a co naopak zakážeme je na nás.

Aktuální přehled balíčků a tříd našeho programu



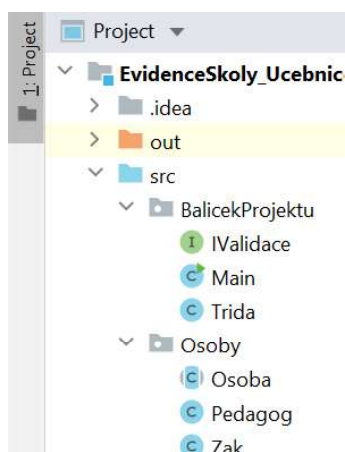
V následující kapitole dokončíme třídu Pedagog a názorně si předvedeme výhody návrhového vzoru enum.

14. Třída Pedagog a využití výčtového typu

Velkou výhodou dědičnosti je, že můžeme z jednoho předka dědit více potomků. **Pozor, nelze z více předků kombinovat jednoho potomka.**

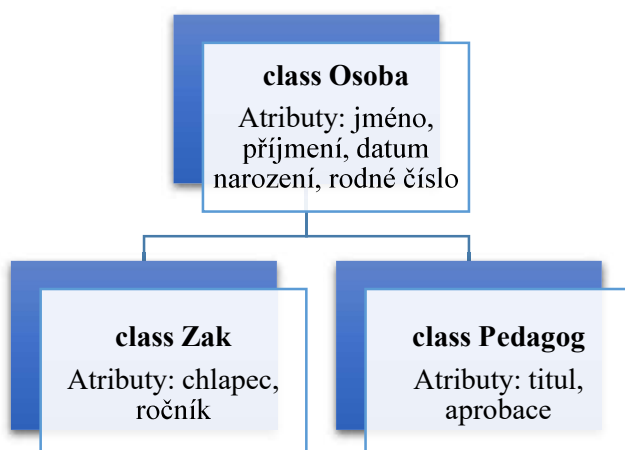
V minulé kapitole jsme naznačili tvorbu nové třídy Pedagog. Jako základ použijeme naši abstraktní třídu Osoba. Třidu samozřejmě vytvoříme v nedávno vytvořeném package Osoby.

Nová třída Pedagog.java



Každý registrovaný pedagog bude mít z Osoby povinné atributy jméno, příjmení, rodné číslo a datum narození. Mezi nové atributy přidáme titul a seznam pro ukládání aprobací.

Dědická struktura použitá v našem programu

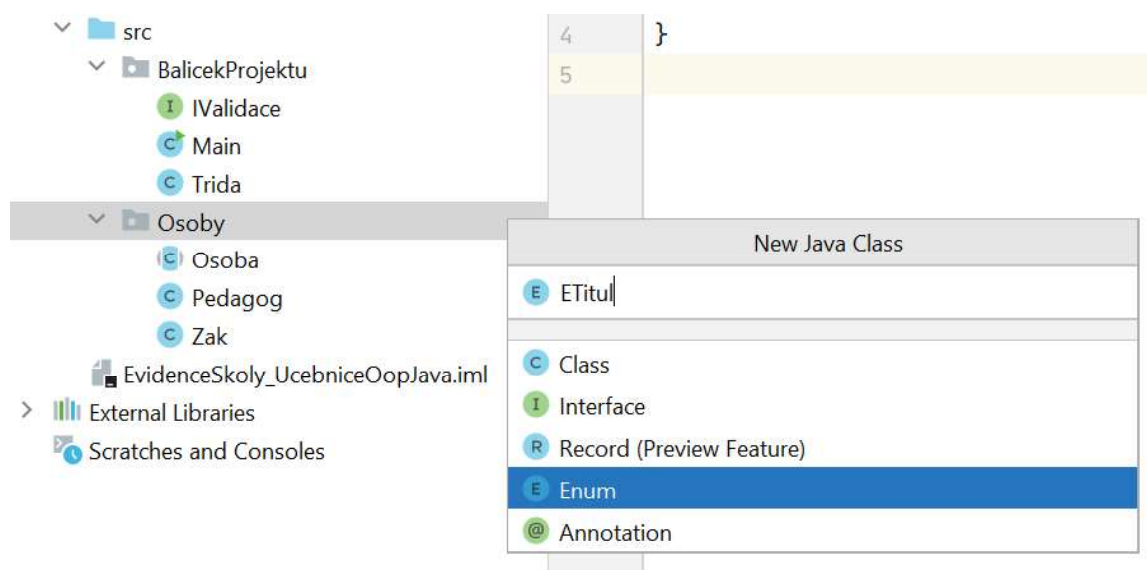


Jako nejjednodušší způsob se jeví titul nadefinovat jako String a aprobace jako List<String>. My ovšem nepůjdeme tou nejjednodušší cestou, nýbrž cestou optimální.

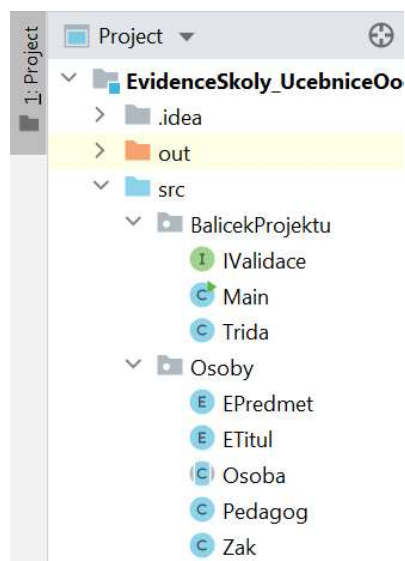
Výčtový typ Enum

Je zřejmé, že titulů existuje pouze konečné omezené množství. Můžeme si je v duchu vyjmenovat. Jakýkoli jiný řetězec, vzniklý například pomocí překlepu, pak není žádoucí. To samé platí pro předměty. Zkratkám jako ČJ nebo PR rozumí každý, text s hodnotou XY by však již smysl nedával. Právě pro tyto případy je vhodné zavést v programu tzv. **výčtový typ**. Protože se naše enumerické datové typy budou týkat třídy Pedagog, vytvoříme je do stejného package Osoby.

Tvorba nového výčtového typu v prostředí IntelliJ



Aktuální přehled balíčků a tříd našeho programu



Kód enum

Naprogramovat enum není žádná věda. Stačí si jen určit, jakých hodnot může náš výčet nabývat. Pozor – pro hodnoty výčtového typu platí jasně daná kritéria. Nesmí například začínat číslem, nesmí obsahovat mezeru či interpunkční znaménka. Dále se můžeme setkat i s požadavkem, aby hodnoty byly vypisovány velkými písmeny. Jak možná víte, velká písmena se v programování používají na určení konstant. A právě výčet konkrétních daných hodnot lze do jisté míry vnímat jako konstanty.

ETitul.java

```
package Osoby;

public enum ETitul {
    MGR,
    ING,
    BC,
    PHDR,
    BEZ_TITULU
}
```

EPredmet.java

```
package Osoby;

public enum EPredmet {
    PR, ČJ, AJ, D, M, ITCH, F, TV, CHE, Z
}
```

Použití enum

Po vytvoření nové class *Pedagog* nejprve dopíšeme část `extends` definující předka v rámci dědičnosti. Následně definujeme titul pomocí enumerického datového typu *ETitul*. Nakonec definujeme seznam s názvem *aprobace*, který konkrétnímu pedagogovi přisoudí množinu předmětů typu *EPredmet*.

Atributy třídy Pedagog (Pedagog.java)

```
package Osoby;
import java.util.ArrayList;

public class Pedagog extends Osoba{
    ETitul titul;
    ArrayList<EPredmet> aprobace;
}
```

Program jsme zabezpečili tak, že není možné do proměnných vkládat libovolné hodnoty (jako tomu bylo u typu String), ale pouze hodnoty definované ve výčtech.

Nyní si pojďme doprogramovat konstruktory. Pedagog opět použije konstruktor předka pomocí klíčového slova **super**. Zde budeme vkládat pouze konkrétní titul a dále alokujeme seznam aprobací. Ten necháme prozatím prázdný.

Konstruktor (Pedagog.java)

```
public Pedagog(String jmeno, String prijmeni, String rodneCislo, LocalDate datumNarozeni, ETitul titul) {  
    super(jmeno, prijmeni, rodneCislo, datumNarozeni);  
    this.titul=titul;  
    aprobace=new ArrayList<EPredmet>();  
}
```

Další metody třídy Pedagog

Pro třídu Pedagog po rozvaze doprogramujeme pár vhodných metod:

- getTitul
- setTitul
- pridejAprobaci – přidá do seznamu aprobací pedagoga další předmět
- vratAprobace – vrátí textový řetězec všech aprobací vyučujícího v závorkách
- vratJmeno – vrátí celé jméno pedagoga včetně titulu
- vratInfo – vrátí informace o pedagogovi včetně jeho aprobací
- toString – vrátí strohé informace o nastavení vnitřních proměnných třídy Pedagog

Metody: getTitul a setTitul

Metody **getTitul**, **setTitul** jsou elementární, s podobnými kódy jsme se již dříve setkali.

Metody getTitul a setTitul (Pedagog.java)

```
public ETitul getTitul() {  
    return titul;  
}  
  
public void setTitul(ETitul titul) {  
    this.titul = titul;  
}
```

Metoda: Přidej aprobaci

Metoda **pridejAprobaci** přidá nový předmět ze vstupu do seznamu. Můžeme ošetřit, že pokud se předmět v seznamu aprobací již vyskytuje, znovu se nepřidá.

Metoda pridejAprobaci (Pedagog.java)

```
public void pridejAprobaci(EPredmet predmet){  
    if (!aprobace.contains(predmet))  
        aprobace.add(predmet);  
}
```

Metoda: Vrať aprobace

Metoda **vratAprobace** tvoří textový řetězec aprobací uložených v seznamu Pedagoga. Text je v závorkách, jednotlivé předměty jsou odděleny čárkou.

Metoda vratAprobace (Pedagog.java)

```
public String vratAprobace() {  
    String text="(";  
    for (int i= 0; i<aprobace.size();i++)  
    {  
        text+=aprobace.get(i);  
  
        //Pokud se nejedná o poslední předmět, oddělíme ve výpisu čárkou  
        if (i!=aprobace.size()-1)  
            text+=", ";  
    }  
    text+=")";  
    return text;  
}
```

Metoda: Vrať jméno

Metody **vratJmeno** nám bude vrátet kompletní jméno učitele, a to včetně titulu. Kód metody je relativně jednoduchý, stejně tak její volání.

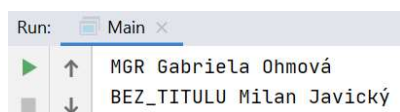
Metoda vratJmeno (Pedagog.java)

```
public String vratJmeno()  
{  
    return String.format("%s %s %s", titul, jmeno, prijmeni);  
}
```

Pozor, třídy *Osoba* / *Zak* / *Pedagog* jsme nedávno přemístili do jiného balíčku, než je aktuálně třída *Main*. Proto, pokud budeme využívat těchto tříd, musíme třídy z balíčku *Osoby* nainportovat. Pokud je naším cílem importovat **všechny** třídy daného balíčku, nejjednodušší je dopsat nahoru do kódu: **Import Osoby.***, kdy znak ***** zastupuje všechny třídy v balíčku..

Volání metody vratJmeno (Main.java)

```
public class Main {  
    public static void main(String[] args) {  
  
        Pedagog p1 = new Pedagog("Gabriela", "Ohmová", "215383/1475", LocalDate.of( year: 1983, month: 3, dayOfMonth: 21), ETitul.MGR);  
        Pedagog p2 = new Pedagog("Milan", "Javický", "110587/1475", LocalDate.of( year: 1987, month: 5, dayOfMonth: 11), ETitul.BEZ_TITULU);  
  
        System.out.println(p1.vratJmeno());  
        System.out.println(p2.vratJmeno());  
    }  
}
```



Z konzolového výstupu je patrné, že metoda funguje, ale výpis není takový, jako bychom si představovali. Vhodnější by byl výpis:

Mgr. Gabriela Ohmová

Milan Javický

ETitul – optimalizace výčtového typu

Pro tyto možnosti však musíme zabrousit nazpátek do enumu *ETitul* a upravit chování výpisu jednotlivých položek. Nejpochopitelnějším způsobem je pro každou jednotlivou položku vytvořit vlastní metodu **toString**. Nejedná se přitom o přetížení metody *toString*. Všimněte si, že jednotlivé metody se v parametrech nijak neliší, takže o přetížení opravdu nejde. Jedná se o speciální způsob práce typický právě pro enumerické datové typy.

Úprava výčtového typu – verze 1 (ETitul.java)

```
package Osoby;

public enum ETitul {
    |  MGR{
    |      @Override
    |      public String toString(){return "Mgr.";}
    |  },
    |  ING{
    |      @Override
    |      public String toString(){return "Ing.";}
    |  },
    |  BC{
    |      @Override
    |      public String toString(){return "Bc.";}
    |  },
    |  PHDR{
    |      @Override
    |      public String toString(){return "Phdr.";}
    |  },
    |  BEZ_TITULU{
    |      @Override
    |      public String toString(){return "";}
    |  }
}
```

Zápis se jeví tak, že jednotlivé položky enumu se samy o sobě jeví jako třídy. Každá má vlastní název a vlastní přepsanou metodu toString. Navíc, pokud jste fanoušky zajímavých syntaxí, můžete (ale nemusíte) náš ETitul modifikovat ještě jinak. Podotýkám, že i přes změnu kódu bude enum pracovat sémanticky stejně, jako se zápisem výše.

Úprava výčtového typu – verze 2 (ETitul.java)

```
package Osoby;

public enum ETitul {

    //volání konstruktoru - tvorba instancí třídy vevnitř téže třídy
    MGR( textTitulu: "Mgr."),
    ING( textTitulu: "Ing."),
    BC( textTitulu: "Bc."),
    PHDR( textTitulu: "Phdr."),
    BEZ_TITULU( textTitulu: "");

    //vnitřní proměnná pro uložení textu reprezentujícího titul
    private String textTitulu;

    //konstruktor objektu ETitul
    private ETitul (String textTitulu){
        this.textTitulu = textTitulu;
    }

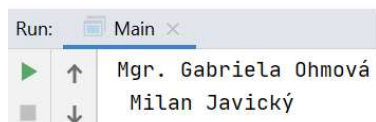
    @Override
    public String toString(){
        return textTitulu;
    }
}
```

Poznámka: Všimněte si velmi netypického kódu, kdy objekty dané třídy tvoříte interně jako součást téže třídy.

Pojďme si ověřit, jak se s modifikovaným enum proměnil výpis metody: **Vrat' jméno pedagoga.**

Volání metody vratJmeno s optimalizovaným výpisem (Main.java)

```
System.out.println(p1.vratJmeno());
System.out.println(p2.vratJmeno());
```



Metoda: Vrat' info o pedagogovi

Následující metoda bude vracet veškeré informace o objektu třídy Pedagog v přehledné formě. Abychom si práci zjednodušili, metoda bude ve svém těle volat již existující metody vratJmeno a vratAprobace.

Metoda vratInfo (Pedagog.java)

```
public String vratInfo()
{
    String text="";
    text+=String.format("%s %s\n", vratJmeno(), vratAprobace());
    text+=getDatumNarozeni();
    return text;
}
```

Z kódu výše je patrné, že metoda `vratInfo` volá ve svém těle dosud nedefinovanou metodu `getDatumNarozeni`. Dosud jsme metodu nepotřebovali, až nyní. Proto ji doprogramujeme. Atribut data narození je spjat se třídou `Osoba`, proto metodu `getDatumNarozeni` doprogramuji právě zde.

Metoda getDatumNarozeni (Osoba.java)

```
public String getDatumNarozeni() {
    return datumNarozeni.format(DateTimeFormatter.ofLocalizedDate(FormatStyle.MEDIUM));
}
```

Poznámka: Obecně je špatně, když se programátoři učí ke každé proměnné tvořit metodu `get` a `set` automaticky. Když vše bezmyšlenkovitě povolíte pro čtení i pro zápis, metody `get` a `set` pozbývají významu. To už můžeme rovnou ke každé proměnné dopsat modifikátor přístupu `public` a vyjde to nastejno. Platí, že existence každé metody by měla mít své jasné opodstatnění.

Pokud chcete, můžete novou metodu `getDatumNarozeni` (společnou pro všechny třídy děděné ze třídy `Osoba`) volat i z metody `vratInfo` třídy `Zak`. Doposud tomu tak nebylo. Stačí upravit původní kód, který se úpravou značně zjednoduší a zkrátí.

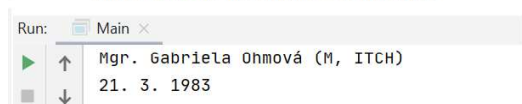
Metoda vratInfo - úprava (Pedagog.java)

```
public String vratInfo()
{
    return String.format("%s %s (%s), %d. ročník", jmeno, prijmeni, getDatumNarozeni(), rocnik);
}
```

Třídu `Pedagog` a její metody máme naprogramovány. Pojdme nyní ověřit funkčnost stávajícího kódu. Vytvoříme nového pedagoga, přidáme mu 2 aprobace a vypíšeme o něm informace na konzoli.

Volání metody vratInfo (Main.java)

```
public class Main {  
    public static void main(String[] args) {  
        Pedagog p1 = new Pedagog("Gabriela", "Ohmová", "215383/1475", LocalDate.of( year: 1983, month: 3, dayOfMonth: 21), ETitul.MGR);  
        p1.pridejAprobaci(EPredmet.M);  
        p1.pridejAprobaci(EPredmet.ITCH);  
  
        System.out.println(p1.vratInfo());  
    }  
}
```



Metoda: toString třídy Pedagog

Pro úplnost doprogramujeme metodu, která obstará základní výpis stavů vnitřních proměnných. Použijeme starou známou metodu **toString**. Nezapomeneme použít klíčové slovo **override**. Metodu vygenerujeme automaticky pomocí klávesové zkratky levý Alt + Insert.

Metoda toString (Pedagog.java)

```
@Override  
public String toString() {  
    return "Pedagog{" +  
        "titul=" + titul +  
        ", apropace=" + apropace +  
        ", jmeno='" + jmeno + '\'' +  
        ", prijmeni='" + prijmeni + '\'' +  
        ", rodneCislo='" + rodneCislo + '\'' +  
        ", datumNarozeni=" + datumNarozeni +  
        '}';  
}
```

Implementace pedagoga jako třídního

Pojďme ještě naposledy zasáhnout do našeho kódu. Pokud zavzpomínáte na programování class Třída, novou třídu jsme tvořili pomocí čísla ročníku a textové proměnné nazvané třídní. Nyní můžeme tento text vhodněji nahradit právě objektem typu Pedagog. Pojďme se podívat, jaké úprav je třeba provést.

Úprava class Trida (Trida.java)

```
package BalicekProjektu;
import Osoby.Zak;
import Osoby.Pedagog;

import java.util.ArrayList;
import java.time.LocalDate;

public class Trida implements IValidace {
    private ArrayList<Zak> zaci;
    private byte rocnik;
    private Pedagog tridni;

    public Trida(byte rocnik, Pedagog tridni)
    {
        this.rocnik = rocnik;
        this.tridni = tridni;
        zaci = new ArrayList<Zak>();
    }

    public String vratInfo()
    {
        String text=System.lineSeparator()+"*****"
            +System.lineSeparator();
        text+=String.format("%d. třída (třídní: %s)%n", rocnik, tridni.vratJmeno());
        text+=vratSeznamZaku();
        return text;
    }
}
```

Ověření funkčnosti

Máme naprogramovánu třídu, pedagoga i žáky. Můžeme vyzkoušet, jak program funguje. Jak spolu jednotlivé třídy kooperují.

Poznámka: Aby program nevypisoval nadbytečné informace, v příkladu níže jsou použita pouze validní rodná čísla.

Testování existujících tříd a metod (Main.java)

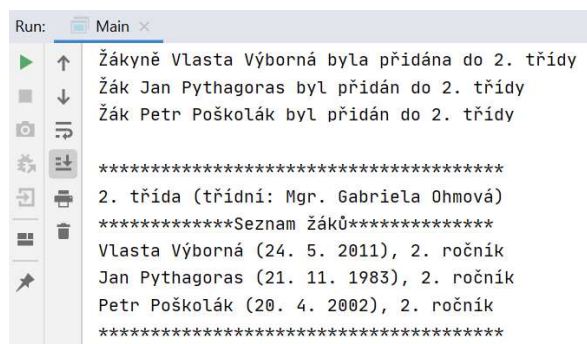
```
import Osoby.*;

public class Main {
    public static void main(String[] args) {
        Pedagog p1 = new Pedagog("Gabriela", "Ohmová", "215383/1475", LocalDate.of( year: 1983, month: 3, dayOfMonth: 21), ETitul.MGR);
        p1.pridejAprobaci(EPredmet.M);
        p1.pridejAprobaci(EPredmet.ITCH);

        Trida t1=new Trida((byte)2,p1);

        System.out.println(t1.pridejNovehoZaka( jmeno: "Vlasta", prijmeni: "Výborná", rodne_cislo: "115524/5652",
        LocalDate.of( year: 2011, month: 5, dayOfMonth: 24), chlapec: false));
        System.out.println(t1.pridejNovehoZaka( jmeno: "Jan", prijmeni: "Pythagoras", rodne_cislo: "831121/4978",
        LocalDate.of( year: 1983, month: 11, dayOfMonth: 21), chlapec: true));
        System.out.println(t1.pridejNovehoZaka( jmeno: "Petr", prijmeni: "Poškolák", rodne_cislo: "020420/1239",
        LocalDate.of( year: 2002, month: 4, dayOfMonth: 20), chlapec: true));

        System.out.println(t1.vratInfo());
    }
}
```



```
Run: Main x
Žákyně Vlasta Výborná byla přidána do 2. třídy
Žák Jan Pythagoras byl přidán do 2. třídy
Žák Petr Poškolák byl přidán do 2. třídy

*****
2. třída (třídní: Mgr. Gabriela Ohmová)
*****Seznam žáků*****
Vlasta Výborná (24. 5. 2011), 2. ročník
Jan Pythagoras (21. 11. 1983), 2. ročník
Petr Poškolák (20. 4. 2002), 2. ročník
*****
```

Bystřejším z Vás jistě neušlo, že jsme do programu nezakomponovali kód ověřující validaci rodného čísla u pedagoga. Problémem je atribut pohlaví, kterou třída Pedagog neobsahuje. Bez této proměnné není možné celkovou validaci provést. Pokud by přesto klient funkcionalitu vyžadoval, není problém příklad poopravit a mírně rozšířit. Případnou implementaci nechám na čtenáři.

Zadání

V této kapitole jsme si mimo jiné ukázali použití výčtového typu enum. Realizace v projektu může probíhat na mnoha rovinách. Pokuste se přijít na další vhodné příklady použití enumerických datových typů v našem systému. Ať už v této fázi budování projektu nebo ve fázi potencionálního rozšíření o další prvky školního systému.

Zalistujte pár stránek nazpět a najděte si kód a výpis aktualizované metody vratJmeno třídy Pedagog. Všimněte si, že v případě výpisu Milana Javického se před jeho jménem objevila mezera. Může za to absence titulu u tohoto pedagoga v souvislosti s pro tento případ neoptimalizovaným kódem metody vratJmeno. Vaším úkolem je tedy upravit tělo metody vratJmeno tak, aby k problémům s mezerou navíc při výpisu nedocházelo.

Prozkoumejte a případně implementujte místo arryListu aprobase (Pedagog.java) objekt typu **EnumSet**.

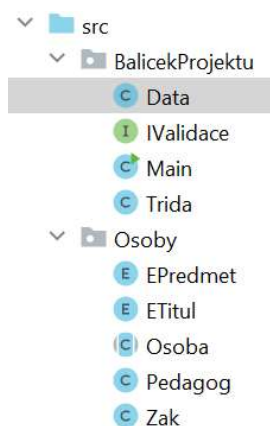
15. Ukládání dat do csv souboru, výjimky

V této kapitole společně vyřešíme ukládání uživatelských dat naší aplikace. Pokusíme se přenést naše žáky, třídy a pedagogy na pevný disk. Bez možnosti ukládání bychom museli data při spuštění zadávat opakovaně při každém spuštění programu. A to opravdu není v zájmu žádného potencionálního uživatele. Nyní se jednoduše veškerá data načtou ze souboru. Jak už z názvu kapitoly vyplývá, jako úložiště použijeme nejjednodušší a zřejmě i nejuniverzálnější textový soubor, a to **csv**.

Class Data

Než začneme, upravíme si náš kód. Není příliš vhodné, aby se nám uživatelská data courala někde po třídě Main. Pro tento účel proto vytvoříme novou třídu – **Data.java**. Část kódu z Main.java zde překopírujeme. Novou třídu Data budeme používat jako **statickou třídu**. Bude tedy existovat pouze jednou – nebudeme z ní vytvářet instance. **Třída se automaticky stává statickou, když všechny její vnitřní prvky jsou rovněž statické.** Existence konstruktoru ve statické třídě postrádá smysl.

Aktuální přehled balíčků a tříd našeho programu



Poznámka 1: Obecně lze říci, že ve třídě Main by mělo být jen pár řádků kódu inicializujících spuštění programu, načtení prvotních dat či základní nastavení. Aktuálně třídu Main používáme na testování, proto je zde množství kódu navíc.

Poznámka 2: Pro označení statické třídy nepoužíváme klíčové slovo static u samotného class (jako je tomu v případě některých jiných programovacích jazyků)

Co se týká objemu ukládaných dat, pro jednoduchost ponecháme možnost uložit jednu třídu a jednoho pedagoga. V případě potřeby uložení většího množství informací lze samozřejmě použít kolekci. Načtení defaultních testovacích dat nám umožní metoda **nactiTestovaciData**.

Nezapomeneme naimportovat třídy z balíčku Osoby a třídu pro práci s lokálním datem.

Statická třída Data a statická metoda nactiTestovaciData (Data.java)

```
package BalicekProjektu;
import Osoby.*;
import java.time.LocalDate;

public class Data {
    static public Pedagog p1;
    static public Trida t1;

    static public void nactiTestovaciData() {
        //do statické proměnné definované výše uložíme nového pedagoga s aprobacemi M a ITch
        p1 = new Pedagog("Gabriela", "Ohmová", "215383/1475", LocalDate.of( year: 1983, month: 3, dayOfMonth: 21), ETitul.MGR);
        p1.pridejAprobaci(EPredmet.M);
        p1.pridejAprobaci(EPredmet.ITCH);

        //do statické proměnné definované výše založíme novou třídu - 2. ročník, třídím bude Pedagog p1
        t1 = new Trida((byte) 2, p1);
    }
}
```

V metodě main třídy Main.java tedy nyní nemáme vůbec nic.. tedy až na řádek, kterým zavoláme načtení testovacích dat. Oproti dřívějšímu stavu se jedná o výrazný posun v našem programu. Třída Main.java by neměla být žádným skladištěm. Sluší jí právě co nejméně zdrojového kódu.

Volání metody nactiTestovaciData (Main.java)

```
package BalicekProjektu;

public class Main {
    public static void main(String[] args) {

        Data.nactiTestovaciData();
    }
}
```

AppData, vytvoření vlastní složky programu

Než začneme svá data ukládat, musíme určit vhodné místo na disku. V dřívějších dobách se data ukládala k instalaci programu na disk C do složky Program Files. Tento způsob se již dnes nepoužívá. Mohou nastat problémy se zápisem na disk C nebo problém s přístupem k datům z různých uživatelských účtů. Podobně můžeme skončit, když data budeme ukládat přímo do složky programu. Nemůžeme tušit, kde uživatel složku překopíruje a zda zde nebude problém se zápisem dat. Kde by však problém nikdy nastat neměl je složka **AppData**. Tato složka je vnořena přímo do struktury uživatelského účtu, stejně jako například složka dokumenty. Vlastník účtu má pro tento prostor garanci přístupu pro čtení i zápis (pokud není složka programu nastavena jinak) a garanci práce s ryze vlastními uživatelskými daty.

Pokud si chceme prohlédnout aktuální obsah složky AppData, není nic jednoduššího, než do levého dolního vyhledávacího okna operačního systému Windows napsat text **%appdata%** a potvrdit Enter. Můžete si prohlédnout, kolik dalších instalovaných programů používá pro ukládání právě tuto složku.

Jak ale programově složku našeho programu vytvořit? Použijeme existující třídu `File`, která nám umožní pracovat právě se soubory a složkami. Jako atribut třídy `Data.java` vytvoříme statický objekt třídy `File`, do nějž uložíme kompletní cestu do naší složky. Důvod je prostý. Kdybychom v rámci programu chtěli vytvářet více souborů, cestu ke kořenovému adresáři máme již zaznamenanou a přístupnou. A ano, objekt třídy `File` nám umožňuje uložit i složku, nikoli pouze soubor.

Nejprve importujeme veškeré třídy z balíčku `java.io`. Následně vytvoříme prostor pro ukládání cesty do složky programu. Protože složka bude jediná a neměnná, použijeme statický objekt třídy `File`. Aby nebylo možno ke složce přistupovat přímo, zvolíme modifikátor přístupu **private**.

Samotnou statickou metodu pojmenujeme **vytvorSlozkuProgramu**. V rámci hlavičky metody je patrný dovětek **throws Exception**. Tato formulka značí, že naše metoda může způsobit výjimku. Práce se soubory a složkami je obecně velmi háklivá na chyby. A chyba evokuje pád programu. Výjimka je však schopna libovolnou chybu pasivně (bez Vašeho bližšího určení) zjistit a eliminovat. Nejedná se tedy o aktivní přístup, kdy si je programátor každé možné dílčí chyby vědom a je schopen přesně na situaci reagovat. Zde opravdu nastavujeme celý proces tak, aby si program byl samostatně schopen s jakoukoli chybou poradit a pádu programu tak zabránit. Princip je jednoduchý – potencionálně problémový kód se umístí do **try** bloku. To značí, že se kód provede tzv. nanečisto – v oddělené části paměti. Teprve když se zjistí, že všechno by proběhlo v pořádku, provede program kód podruhé, již načisto. V příkladu jsem zvolil strategii, kdy **try** není součástí metody, ale používám jej u volání metody. Obě varianty jsou možné.

Možná Vás napadne, proč nedat do **try** bloku celý kód. Tato strategie není vůbec efektivní z hlediska využívání systémových zdrojů. Kdybyste každou operaci měli dělat „na dvakrát“, bude si to rovněž žádat vyšší hardwarové prostředky. Proto **try** bloku nenadužívejme. Obecně se výjimky řeší primárně u práce se vstupy a výstupy jakéhokoli typu. Práce se soubory, databázi, uživatelskými vstupy, se sítí, apod.

Metoda vytvorSlozkuProgramu pro vytvoření složky v AppData (Data.java)

```
package BalicekProjektu;
import Osoby.*;
import java.time.LocalDate;
import java.io.*;

public class Data {
    static public Pedagog p1;
    static public Trida t1;

    static private File slozkaProgramu;

    static public void vytvorSlozkuProgramu() throws Exception {
        slozkaProgramu = new File(System.getenv( name: "APPDATA") + File.separator + "Evidence školy");
        if (!slozkaProgramu.isDirectory())
            slozkaProgramu.mkdirs();
    }
}
```

Voláním metody **System.getenv("APPDATA")** získáváme textový řetězec právě na složku aplikačních dat. V mém případě získávám řetězec: **C:\Users\marti\AppData\Roaming**. Příkaz pokračuje přidáním

opačného lomítka a názvu složky „Evidence školy“. Po zavolání metody z Main.java můžeme ověřit, že pokud složka dříve neexistovala, nyní už ji máme k dispozici.

Volání metody vytvorSlozkuProgramu (Main.java)

```
public class Main {  
    public static void main(String[] args) {  
  
        Data.nactiTestovaciData();  
  
        try {  
            Data.vytvorSlozkuProgramu();  
        }  
        catch(Exception e)  
        {  
            System.err.println("Složka nemohla být vytvořena");  
        }  
    }  
}
```

V kódu lze vidět, jak volání metody je uvozeno právě do try bloku. Následně můžeme vidět i **catch** blok, kde je uvedena reakce na případně způsobenou výjimku. V našem případě se na konzoli vypíše textová chybová zpráva. Můžeme ale klidně vzniklý problém např. logovat. Konkrétní vzniklá chyba je pak uložena v parametru e.

Tvorba csv souboru

Další metoda bude velmi podobná té předchozí. Tentokrát budeme vytvářet csv soubor. A to v naší složce programu. Opět si zkonstruujeme privátní objekt typu File a tentokrát do něj uložíme cestu již ke konkrétnímu souboru. V našem případě bude mít název žáci.csv. Uložený název nám následně urychlí přístup k danému souboru.

Rovněž nechybí upozornění na možnou výjimku **throws IOException**, kterým říkáme, že během používání metody může nastat chyba týkající vstup-výstupní operace.

Tělo metody je jednoduché – pokud soubor neexistuje, vytvoří se pomocí **createNewFile()**.

Metoda vytvorSouborZaci pro vytvoření csv souboru v AppDatech (Data.java)

```
static private File souborZaci;  
  
static public void vytvorSouborZaci() throws IOException{  
    souborZaci = new File(slozkaProgramu+File.separator+"žáci.csv");  
    if (!souborZaci.exists())  
        souborZaci.createNewFile();  
}
```

Obdobně jako v předchozím případě voláme i tuto metodu.

Volání metody vytvořSouborZaci (Main.java)

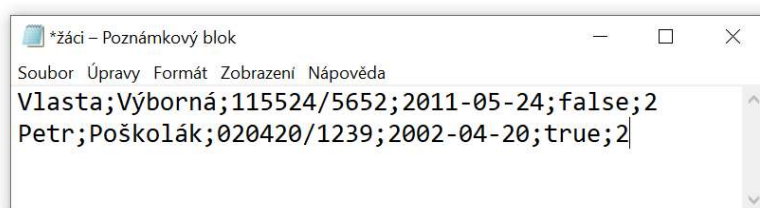
```
try {  
    Data.vytvorSouborZaci();  
}  
catch(Exception e)  
{  
    System.err.println("Soubor nemohl být vytvořen");  
}
```

Csv soubor - formátování

Odpovězme si nyní na otázku, co je to vlastně csv soubor. Jedná se o **textový (String) soubor obsahující tabulková data**. V našem případě interpretujeme veškeré atributy žáka jako jeden řádek tabulky. Jako oddělovač tabulkových dat poslouží speciální symbol, nejčastěji ";". Hodnoty z řádku tabulky musíme číst ve stejném pořadí, v jakém byla zapsána.

V našem případě by obsah csv souboru po zápisu dvou testovacích žáků vypadal následovně.

Obsah souboru žáci.csv v AppDatech



Zápis dat do csv souboru

Abychom si celý proces výroby csv souboru zjednodušili, vytvoříme v rámci třídy Zak metodu vracející textovou csv reprezentaci žáka. Pojmenujeme ji **toCsv()**. Celá situace je zdánlivě komplikovaná, neboť Zak dědí některé atributy ze třídy Osoba. Záznam se tedy skládá z atributů dvou různých tříd. Naštěstí používáme u proměnných předka modifikátor přístupu **protected**. Tedy v rámci dědičnosti jsou proměnné přístupné i pro všechny potomky. Jako oddělovací symbol používáme dříve dohodnutý středník.

Metoda toCsv (Zak.java)

```
public class Zak extends Osoba {  
    private boolean chlapec;  
    private byte rocnik;  
  
    public String toCsv(){  
        return String.format("%s;%s;%s;%s;%s;%s", jmeno, prijmeni, rodneCislo, datumNarozeni, chlapec, rocnik);  
    }  
}
```

Samotný zápis dat do souboru budeme řešit v class Data.java. Problematiku rozdělíme do dvou oddělených metod:

- **csvZaznamyZaci()** – metoda vytvoří a vrátí textovou reprezentaci všech žáků uložených v arraylistu zaci z class Trida.java. Připraví tedy kompletní text pro finální zápis do csv souboru.
- **ulozZakyDoCsv()** – převezme text připravený ze třídy csvZaznamyZaci() a zapíše jej do csv souboru, jehož název je uveden v proměnné souborZaci. Předpokladem pro zápis je existence tohoto souboru.

Metoda csvZaznamyZaci (Data.java)

```
static public String csvZaznamyZaci(){
    String csvZaznamy="";
    //Cyklem foreach procházíme arrayList žáků
    //abychom se k němu dostali, potřebujeme pro něj v class Trida vytvořit getter
    for(Zak zak : t1.getZaci()){
        //Na každém žákovi v arryListu zavoláme metodu, která vrátí jeho csv reprezentaci
        //tu připišeme do proměnné a odřádkujeme.
        csvZaznamy+=zak.toCsv()+"\n";
    }
    return csvZaznamy;
}
```

K arraylistu žáků je potřeba dostat se skrze metodu get. Je totiž v jiné třídě a je privátní. Dopíšeme tedy getter a kód již nebude vykazovat chyby. Tento kód můžeme rovněž automaticky vygenerovat pomocí klávesové zkratky levý alt + insert.

Metoda getZaci (Trida.java)

```
public ArrayList<Zak> getZaci() {
    return zaci;
}
```

Posledním krokem je zápis csv záznamu do souboru. K tomu použijeme dvě třídy, a to **BufferedWriter** a **FileWriter**.

Metoda ulozZakyDoCsv (Data.java)

```
static public void ulozZakyDoCsv() throws IOException {  
    //otevření csv souboru pro zápis  
    try(BufferedWriter bw = new BufferedWriter(new FileWriter(souborZaci,false))) {  
        //do souboru zapíšeme csv záznamy získané metodou csvZaznamyZaci()  
        bw.write(csvZaznamyZaci());  
        //před zavřením souboru vynutíme přepis veškerých dat z vyrovnávací paměti  
        bw.flush();  
    }  
}
```

Blok try, do kterého zapisujeme instanci zápisu do csv souboru nám zjednodušuje programování. Konstrukce **try-with-resources** nám totiž nahrazuje bloky try a finally. Zápis do souboru je tedy ošetřen a finally nám zase na závěr soubor automaticky zavře. Zavření souboru je nezbytná operace. Pokud necháte soubor omylem otevřený pro jeden proces, nemůžete jej pak otevřít v procesu jiném.

Při volání metody z Main.java doplníme blok **catch** pro reakci na vyvolanou výjimku. Ten totiž konstrukce try-with-resources neobsahuje. V případě zájmu můžeme přes argument e vypsát i informaci o tom, k jaké přesně výjimce došlo.

Volání metody ulozZakyDoCsv (Main.java)

```
try {  
    Data.ulozZakyDoCsv();  
}  
catch(Exception e)  
{  
    System.err.println("Data nemohla být uložena: "+e.getMessage());  
}
```

Načtení dat z csv souboru

Při startu programu se nám bude jistě hodit možnost načíst žáky z csv souboru. K tomu použijeme dvě třídy - **BufferedReader** a **FileReader**.

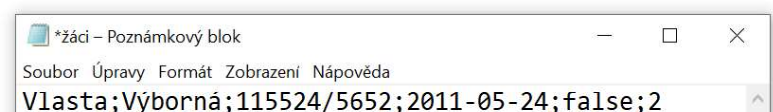
Postup:

- Načíst csv soubor s daty.
- Z každého csv řádku vytvořit objekt třídy Zak.
- Každý takový objekt přemístit do arrayListu žáků.

Kdybychom načítali csv data v průběhu práce s aplikací, mohlo by se stát, že se nám žáci v `arrayListu` duplikují. Před samotným nahráním dat je tedy záhodno `arrayList` kompletně smazat. V našem případě ovšem můžeme tento krok vynechat. Námi definovaná metoda `pridejZaka` totiž žáka přidá žáka pouze v momentě, kdy se jeho rodné číslo v seznamu ještě nevyskytuje. Tedy k duplicitě dat při použití metody nemůže dojít.

Jak bylo psáno výše, z každého řádku souboru bude vytvořen nový žák. Naše csv struktura jako oddělovač jednotlivých buněk používá středník. Pro komfortnější práci použijeme existující metodu **`split`**, která je schopna nám rozdělit řádek do stringového pole právě dle zvoleného oddělovače.

Metoda `split` – princip fungování



Index	0	1	2	3	4	5
Hodnota	"Vlasta"	"Výborná"	"115524/5652"	"2011-05-24"	"false"	"2"

Pozor, záznamy v poli jsou datového typu `String`. Proto je nutné netextové atributy objektu vždy vhodně přeparsovat.

Metoda nactiZakyZCsv (Data.java)

```
static public void nactiZakyZCsv() throws IOException{

    //Otevření csv souboru pro čtení
    try(BufferedReader br = new BufferedReader(new FileReader(souborZaci))){
        String radek;
        //Načítání řádků tak dlouho, dokud nejsme na konci souboru
        while ((radek=br.readLine())!=null) {

            //Z každého řádku vytvoř Stringové pole (csv oddělovačem je středník)
            String[] pole = radek.split(regex: ";");

            //pole má na 0. pozici jméno, na 1. pozici příjmení, na 2. pozici rodné číslo
            //na 3. pozici datum narození, na 4. pozici informaci o pohlaví
            String jmen= pole[0];
            String prijn=pole[1];
            String rodneC = pole[2];
            LocalDate datumNar = LocalDate.parse(pole[3]);
            boolean chlap = Boolean.parseBoolean(pole[4]);

            //Na základě zjištěných hodnot vytváříme a přidáváme nového žáka
            t1.pridejNovehoZaka(jmen, prijn, rodneC, datumNar, chlap);
        }
    }
}
```

Metodu voláme obdobně, jako metodu pro ukládání žáků.

Volání metody nactiZakyZCsv (Main.java)

```
try {
    Data.nactiZakyZCsv();
}
catch(Exception e)
{
    System.err.println("Data nemohla být načtena: "+e.getMessage());
}
```

Otestování

Veškeré metody pro práci se soubory máme funkční. Můžeme je tedy otestovat. Přejdeme do Main.java a volání jednotlivých metod dáme do pořadí vhodného pro testování:

1. Vytvoříme defaultní složku aplikace v app datech. (pokud neexistuje).
2. Vytvoříme csv soubor pro ukládání žáků (pokud neexistuje).
3. Načteme defaultní testovací data programu do operační paměti.
4. Načteme žáky z csv do arrayListu (bud' tam nějací žáci jsou, nebo nikoli).
5. Na konzoli zobrazíme aktuální informace o třídě.
6. Přidáme do arrayListu 2 nové žáky.
7. Žáky uložíme z operační paměti do csv souboru.
8. Program vypneme a zase zapneme. Noví žáci by se měli načíst a zobrazit ve výpisu na konzoli v rámci kroku č. 5

Testování kódu pro práci s csv souborem (Main.java)

```
public class Main {
    public static void main(String[] args) {

        //1
        try {
            Data.vytvorSlozkuProgramu();
        }
        catch(Exception e) {
            System.err.println("Složna nemohla být vytvořena");
        }

        //2
        try {
            Data.vytvorSouborZaci();
        }
        catch(Exception e) {
            System.err.println("Soubor nemohl být vytvořen");
        }

        //3
        Data.nactiTestovaciData();

        //4
        try {
            Data.nactiZakyZCsv();
        }
        catch(Exception e) {
            System.err.println("Data nemohla být načtena: "+e.getMessage());
        }

        //5
        System.out.println(Data.t1.vratInfo());

        //6
        System.out.println(Data.t1.pridejNovehoZaka( jmeno: "Vlasta",   prijmeni: "Výborná",   rodne_cislo: "115524/5652",
            LocalDate.of( year: 2011,   month: 5,   dayOfMonth: 24),   chlapec: false));
        System.out.println(Data.t1.pridejNovehoZaka( jmeno: "Petr",   prijmeni: "Poškolák",   rodne_cislo: "020420/1239",
            LocalDate.of( year: 2002,   month: 4,   dayOfMonth: 20),   chlapec: true));

        //7
        try {
            Data.ulozZakyDoCsv();
        }
        catch(Exception e) {
            System.err.println("Data nemohla být uložena: "+e.getMessage());
        }

        //8
        //Program vypneme a zase zapneme. Noví žáci by se měli načíst a zobrazit ve výpisu na konzoli v rámci kroku č. 5
    }
}
```

Výpisy na konzoli při 1. a 2. spuštění programu

```
Main x
*****
2. třída (třídní: Mgr. Gabriela Ohmová)
*****Seznam Žáků*****
*****

Žákyně Vlasta Výborná byla přidána do 2. třídy
Žák Petr Poškolák byl přidán do 2. třídy
```

```
Main x
*****
2. třída (třídní: Mgr. Gabriela Ohmová)
*****Seznam Žáků*****
Vlasta Výborná (24. 5. 2011), 2. ročník
Petr Poškolák (20. 4. 2002), 2. ročník
*****

Chyba přidání nového žáka: Bylo zjištěno, že žák/žákyně s rodným číslem 115524/5652 již ve třídě existuje.
Chyba přidání nového žáka: Bylo zjištěno, že žák/žákyně s rodným číslem 020420/1239 již ve třídě existuje.
```

Jak je patrné z výpisů, při prvním spuštění program ještě žádné žáky neobsahuje. Při druhém spuštění si pak hned na začátku 2 žáky načte z csv souboru. Když se tedy pokusíme znovu přidat tytéž žáky z Main.java, program nám to nedovolí. Na základě rodného čísla zjistí duplicitu a vypíše informaci o chybě.

Program tedy funguje. Naše sekvence příkazů v Main je ale natolik direktivní, že není možné tento postup aplikovat v klasickém programu. Uživatel si chce vybrat, zda nyní bude zakládat žáka, zda bude ukládat žáky do souboru, prostě kdy provede kterou operaci. V poslední kapitole se tedy naučíme vytvořit uživatelské rozhraní našeho programu.

16. Uživatelské rozhraní programu

Doposud jsme veškeré funkce programu testovali psaním příkazů do metody Main. Uživatel zde ovšem nic psát nemůže. To je prostor programátora. Situaci vyřešíme tvorbou jednoduchého textového ovládání. Náš program konečně ožije. Algoritmizace této kapitoly nebude ani tak složitá, jako spíše zdoluhavá. Kdykoli píšeme kód komunikující s uživatelem, bude obsahovat množství textu.

Definování klíčových příkazů

Na začátku je nezbytné si určit, které metody dovolíme uživateli používat. Vybereme všechny pro uživatele důležité metody a ke každé z nich určíme příkaz – **klíčové slovo pro zavolání dané metody**.

V rámci našeho programu jsem si určil následující příkazy:

- new - vytvoř nového žáka
- del - smaž stávajícího žáka
- show - zobraz informace o třídě
- save – ulož žáky do csv souboru
- help - zobraz nápovědu programu
- exit - ukonči program

Samozřejmě platí, že čím více funkcí bude program poskytovat, tím větší portfolio příkazů je třeba nabídnout.

Poznámka: Příkaz open pro načtení žáků z csv souboru vytvářet nebudeme. V kódu vyřešíme situaci tak, že načítání žáků proběhne automaticky se spuštěním programu.

Další krokem bude vytvoření třídy speciálně určené pro obsluhu našich příkazů. Pojmenoval jsem ji originálně – `UzivatelkeRozhrani.java`. Z této třídy rovněž není nutné vytvářet instance. Jedná se pouze o shluk několika metod, žádný originální atribut se zde nevyskytuje. Proto jsem se při programování přiklonil k vytvoření statické třídy. Není to ale nezbytné.

Práce s uživatelskými daty

Než se dostaneme k definování jednotlivých příkazů, zajistíme vytvoření csv souboru pro ukládání žáků (pokud neexistuje), případně načtení dat ze souboru a načtení testovacích dat. Kódy již máme z předchozí kapitoly hotové ve třídě Main. Nyní algoritmy přesuneme do speciálních statických metod právě v rámci třídy UzivatelскеRozhrani. V Main by neměl zůstat téměř žádný kód.

Třída UzivatelскеRozhrani bude obsahovat pouze 2 public metody, tou první je metoda pro načtení dat.

Metoda nactiData (UzivatelскеRozhrani.java)

```
public static void nactiData()
{
    //volání metody pro vytvoření programové složky a souboru pro ukládání žáků (pokud neexistují)
    vytvorSouboryASlozkyProgramu();

    //načtení testovacích dat (vytvoření třídy a třídního pedagoga)
    Data.nactiTestovaciData();

    //TESTOVACÍ DATA - přidání 2 testovacích žáků, posléze tyto řádky kódu smazat!!
    Data.t1.pridejNovehoZaka( jmeno: "Vlasta", prijmeni: "Výborná", rodne_cislo: "115524/5652",
        LocalDate.of( year: 2011, month: 5, dayOfMonth: 24), chlapec: false);
    Data.t1.pridejNovehoZaka( jmeno: "Petr", prijmeni: "Poškoťák", rodne_cislo: "020420/1239",
        LocalDate.of( year: 2002, month: 4, dayOfMonth: 20), chlapec: true);
    System.out.println("Testovací data byla v pořádku načtena");

    //načtení žáků z csv souboru (pokud existují)
    try {
        Data.nactiZakyZCsv();
        System.out.println("Data ze souboru byla v pořádku načtena");
    }
    catch(Exception e) {
        System.err.println("Data ze souboru nemohla být načtena: "+e.getMessage());
    }
}
```

Na prvním řádku nám metoda volá jinou metodu, kterou ještě nemáme nadefinovanou, a to vytvorSouboryASlozkyProgramu. Tato už může být označena modifikátorem přístupu private, neboť se bude nacházet ve stejné třídě, ze které je sama volána. Metoda v případě potřeby vytvoří složkovou strukturu programu v aplikačních datech.

Metoda vytvořSouboryASlozkyProgramu (UzivatskeRozhrani.java)

```
private static void vytvořSouboryASlozkyProgramu()
{
    //tvorba složky programu (pokud neexistuje)
    try {
        Data.vytvorSlozkuProgramu();
    }
    catch(Exception e) {
        System.err.println("Složna nemohla být vytvořena");
    }

    //tvorba csv souboru pro ukládání žáků (pokud neexistuje)
    try {
        Data.vytvorSouborZaci();
    }
    catch(Exception e) {
        System.err.println("Soubor nemohl být vytvořen");
    }
}
```

Předchozí 2 metody jsou na sebe navázané. Stačí je tedy zavolat jediným příkazem. To už zařídíme z metody Main. Neboť se jedná o statickou třídu, nebudeme vytvářet instanci, ale metodu zavoláme přímo přes příslušnou třídu.

Volání metody nactiData (Main.java)

```
public class Main {
    public static void main(String[] args) {

        UzivatelskeRozhrani.nactiData();
    }
}
```

Vykonávání uživatelských příkazů

Druhou (a poslední) public metodou bude metoda pro čtení klíčových příkazů z console s názvem startUI. Vše v programu se bude od nyníška dít pouze skrze tuto metodu. Program nás při startu uživatelského rozhraní nejprve přivítá, následně čte příkazy uživatele neustále dokola v cyklu, a to tak dlouho, dokud uživatel nenapíše slovo exit. Protože řešíme consolový vstup, je nasnadě v naší třídě definovat scanner.

Metoda startUI (UzivatskeRozhrani.java)

```
package BalicekProjektu;
import java.time.LocalDate;
import java.util.Scanner;

public class UzivatelskeRozhrani {

    static private Scanner sc = new Scanner(System.in);

    public static void startUI(){
        System.out.println("-----");
        System.out.println("Vítejte v programu Evidence školy!");
        System.out.println("-----");
        String prikaz="";
        do {
            System.out.print("Příkaz: ");
            prikaz = sc.nextLine().toLowerCase().trim();
            vykonejPrikaz(prikaz);
            System.out.println("-----");
        }while(!prikaz.equals("exit"));
    }
}
```

*Poznámka: Na každém příkazu se po přečtení z console zavolají 2 metody. Metoda **toLowerCase** nám řetězec převede na malá písmena, i když bylo původně zapsáno např. při aktivované funkci CapsLock. Metoda **trim** zase smaže omylem napsané mezery před a za řetězcem. Nesmaže mezerníky mezi slovy. Obě metody nám eliminují chyby uživatele při zadávání příkazů do console.*

Poznámka 2: Volaná metoda vykonejPrikaz ještě neexistuje, doprogramujeme ji vzápětí.

Uživatelské rozhraní spustíme jednoduše tak, že zavoláme metodu startUI ze třídy Main.

Metoda main (Main.java)

```
package BalicekProjektu;

public class Main {
    public static void main(String[] args) {

        UzivatelskeRozhrani.nactiData();
        UzivatelskeRozhrani.startUI();
    }
}
```

A to je prosím celá naše třída Main. Více již toho zde nenajdete. Veškeré vstup/výstupní kódy nyní dopíšeme primárně do třídy UzivatelskeRozhrani.

Metoda: Vykonej příkaz

Vraťme se k již zmíněné metodě **vykonejPrikaz**. Ta nás na základě textového příkazu na vstupu bude schopna přepnout na příslušnou metodu. Výstup nemá metoda žádný.

Metoda vykonejPrikaz (UzivatelскеRozhrani.java)

```
private static void vykonejPrikaz(String prikaz) {  
    {  
        switch (prikaz) {  
            case "new":  
                newPrikaz();  
                break;  
            case "del":  
                delPrikaz();  
                break;  
            case "show":  
                System.out.println(Data.tl.vratInfo());  
                break;  
            case "save":  
                savePrikaz();  
                break;  
            case "help":  
                helpPrikaz();  
                break;  
            case "exit":  
                exitPrikaz();  
                break;  
            default:  
                System.out.println("Chybný příkaz na vstupu, pokračujte zadání příkazu help.");  
                break;  
        }  
    }  
}
```

U metod tohoto typu je dobrým zvykem používat rozcestník, neboli switch, který je schopen nám nahradit vícenásobné větvení. Kód je přehlednější.

Z kódu voláme metody, které jsme dosud nedefinovali. To nás teprve čeká. Ale jako kostra uživatelského rozhraní nám switch slouží výborně.

Rovněž je zde přítomný blok default, který se využije, když uživatelem zadané slovo neodpovídá žádnému z podporovaných příkazů. Necháme prostě vypsát informaci, že nápověda programu se zobrazí po zadání klíčového slova help.

Příkaz - Nápověda

Když už jsme u toho, pojďme se zaměřit právě na statickou metodu **helpPrikaz**. Metoda tohoto typu by měla být součástí každého uživatelského rozhraní. Výpis použitelných příkazů se může hodit za různých okolností. Jak sami vidíte, jedná se pouze o výstupní informace. Žádná pořádná algoritmizace nás zde nečeká.

Metoda helpPrikaz (UzivatskeRozhrani.java)

```
static private void helpPrikaz() {  
    System.out.println("Seznam platných příkazů pro program Evidence školy:");  
    System.out.println("\tnew \tvytvoř nového žáka");  
    System.out.println("\tdel \tsmaž stávajícího žáka");  
    System.out.println("\tshow\tzobraz informace o třídě");  
    System.out.println("\tsave\tulož žáky ve třídě");  
    System.out.println("\thelp\tzobraz nápoovědu programu");  
    System.out.println("\texit\tukončení programu");  
}
```

Příkaz - Nový žák

Asi nejsložitější metodou se jeví metoda **newPrikaz**, která se v programu vykoná po zadání klíčového slova **new**.

Obsluhuje veškerou komunikaci s uživatelem týkající se vytvoření nového žáka. Je tedy plná externích vstupů od uživatele. Což samozřejmě vede k množství potencionálních chyb. Musíme vždy přepokládat, že uživatel je člověk chybující a ne každý zadaný údaj bude validní. K chybám úplně stačí přepnutá anglická klávesnice, vypnutá numerická klávesnice, nepozornost či překlep. Z tohoto důvodu je potřeba potencionální chyby detekovat. Buď (např. pomocí if) cílíme na konkrétní chybu – tomu říkáme **aktivní ošetřování chyb**, nebo pomocí **try-catch bloku** vychytáváme veškeré možné problémy našeho kritického kódu. Tomu říkáme **pasivní ošetřování chyb**. Nejčastěji se setkáme s kombinací, většina problémů se však vychytává pasivním přístupem.

Poznámka: Používání try-catch bloku je velmi snadné a svádí k nadužívání. Dejte však pozor, pasivní šetření chyb vede ke zvýšené spotřebě hw zdrojů, program bude klást zvýšené nároky na konfiguraci počítače. Try-catch blokem uvozujte pouze kód, u kterého je to nezbytné.

Metoda newPrikaz (UzivatelскеRozhrani.java)

```
static private void newPrikaz() {
    System.out.println("Nový žák:");

    //aktivní ošetření vstupu - pohlaví
    //pokud pohlaví nebude určeno písmenem a nebo b, metoda se ukončí a žák nebude vytvořen
    System.out.print("\tZadej pohlaví: a - chlapec, b - dívka: ");
    String pohlaviS = sc.nextLine().trim().toLowerCase();
    boolean chlapec=true;
    if(pohlaviS.equals("a"))
        chlapec= true;
    else if (pohlaviS.equals("b"))
        chlapec = false;
    else {
        System.out.println("\nChyba na vstupu, žák nebude vytvořen");
    }

    //U Stringu netřeba ošetřovat chyby, textem může být cokoli
    System.out.print("\tJméno: ");
    String jmeno = sc.nextLine().trim();
    System.out.print("\tPříjmení: ");
    String prijmeni = sc.nextLine().trim();
    System.out.print("\tRodné číslo (formát xxxxxx/xxxx): ");
    String rc = sc.nextLine().trim();

    //pasivní ošetření vstupu - ostatní
    try{
        System.out.print("\tDen narození: ");
        int den = sc.nextInt();
        System.out.print("\tMěsíc narození (číslo měsíce): ");
        int mesic = sc.nextInt();
        System.out.print("\tRok narození: ");
        int rok = sc.nextInt();

        //Pokud výjimka nezpůsobí chybu, nový žák bude na konci metody vytvořen a přidán do třídy
        System.out.println(Data.t1.pridejNovehoZaka(jmeno, prijmeni, rc, LocalDate.of(rok, mesic, den),chlapec));
    }
    catch (Exception e) {
        System.out.println("\n Chyba na vstupu, žák nebude vytvořen");
    }
}
```

Pokud vstupy budou shledány validními, kód úspěšně projde až k volání metody **pridejNovehoZaka**. Co když jsme ale udělali chybu a žák ve třídě již existuje? Bez starostí, tento stav jsme již dříve řešili právě v rámci metody pro přidání žáka.

Příkaz - Smaž žáka

Podobně přistupujme i k další chybějící metodě uživatelského rozhraní, která maže žáka ze třídy. Její kód je jednoduchý. Vnitřní logika už totiž byla dávno naprogramována v rámci metody **odeberZaka**.

Metoda delPrikaz (UzivatskeRozhrani.java)

```
private static void delPrikaz() {  
    System.out.print("Smazání žáka s rodným číslem (xxxxxx/xxxx):");  
    System.out.println(Data.t1.odeberZaka(sc.nextLine()));  
}
```

Příkaz – Ukaž třídu

Další metoda není vůbec složitá. Jejím úkolem bude pouze vypsát na consoli informace o třídě, a to pomocí již vytvořené metody **vratInfo()**. Budeme tak mít přehled o pohybu žáků v naší třídě.

Metoda showPrikaz (UzivatskeRozhrani.java)

```
private static void showPrikaz() {  
    System.out.println(Data.t1.vratInfo());  
}
```

Příkaz – Ulož žáky

Naše předposlední metoda si klade za cíl uložit žáky do csv souboru. Opět bude volat již existující metodu, tentokrát **ulozZakyDoCsv()** ze třídy Data.java.

Práce se souborem je potenciálně nebezpečná operace, proto zápis uzavřeme do try – catch bloku.

Metoda savePrikaz (UzivatskeRozhrani.java)

```
private static void savePrikaz() {  
    try {  
        Data.ulozZakyDoCsv();  
        System.out.println("Data byla v pořádku uložena.");  
    } catch (Exception e) {  
        System.err.println("Data nemohla být uložena: " + e.getMessage());  
    }  
}
```

Příkaz – Ukonči program

Poslední programovaná metoda bude reakcí na příkaz exit. Jak vyplývá se samotného názvu, klade si za cíl ukončit program. Může se stát, že uživatel zapomene data uložit, proto mu tuto skutečnost během zavírání programu pro jistotu připomeneme.

Metoda `exitPrikaz` (`UzivatelскеRozhrani.java`)

```
static private void exitPrikaz(){
    System.out.print("Přejete si uložit data? (a): ");
    String prikaz = sc.nextLine().trim().toLowerCase();
    if(prikaz.equals("a"))
        savePrikaz();
    System.out.println("Program bude nyní ukončen..");
}
```

Klíčové slovo **exit** zároveň ukončí cyklus z metody **startUI**. A protože se jedná o poslední volanou metodu celého programu, ten se vzápětí ukončí.

Gratuluji, právě jste dokončili textové uživatelské rozhraní. Byla to hromada kódu, ale nejednalo se o žádnou složitou algoritmizaci. Pojdme si ovládání programu užít!

Ukázka práce s textovým uživatelským rozhraním v konzoli

```
Testovací data byla v pořádku načtena
Data ze souboru byla v pořádku načtena
-----
Vítejte v programu Evidence školy!
-----
Příkaz: help
Seznam platných příkazů pro program Evidence školy:
    new      vytvoř nového žáka
    del      smaž stávajícího žáka
    show     zobraz informace o třídě
    save     ulož žáky ve třídě
    help     zobraz nápoovědu programu
    exit     ukončení programu
-----
Příkaz: show

*****
2. třída (třídní: Mgr. Gabriela Ohmová)
*****Seznam žáků*****
Vlasta Výborná (24. 5. 2011), 2. ročník
Petr Poškolák (20. 4. 2002), 2. ročník
*****

-----

Příkaz: new
Nový žák:
    Zadej pohlaví: a - chlapec, b - dívka: b
    Jméno: Jana
    Příjmení: Kuchařová
    Rodné číslo (formát xxxxxx/xxxx): 036120/3326
    Den narození: 20
    Měsíc narození (číslo měsíce): 11
    Rok narození: 2003
Žákyně Jana Kuchařová byla přidána do 2. třídy
-----
```

Ukázka práce s textovým uživatelským rozhraním v konzoli - pokračování

Příkaz: *show*

```
*****
2. třída (třídní: Mgr. Gabriela Ohmová)
*****Seznam žáků*****
Vlasta Výborná (24. 5. 2011), 2. ročník
Petr Poškolák (20. 4. 2002), 2. ročník
Jana Kuchařová (20. 11. 2003), 2. ročník
*****
```

Příkaz: *del*

```
Smazání žaka s rodným číslem (xxxxxx/xxxx):115524/5652
Žákyně Vlasta Výborná byla odstraněna z 2. třídy
```

Příkaz: *show*

```
*****
2. třída (třídní: Mgr. Gabriela Ohmová)
*****Seznam žáků*****
Petr Poškolák (20. 4. 2002), 2. ročník
Jana Kuchařová (20. 11. 2003), 2. ročník
*****
```

Příkaz: *exit*

```
Přejete si uložit data? (a): a
Data byla v pořádku uložena.
Program bude nyní ukončen..
```

Závěr

Dospěli jste až sem, můžete si blahopřát. Program je funkční. Jistě, ještě to není na nasazení do ostrého provozu. To ale nebylo cílem. Důležité je porozumět základním strukturám OOP a pochopit budování OOP programu. Přemýšlet, co je vhodné implementovat a jak to implementovat. Program se neustále rozšiřuje a mění. Vaším úkolem je tvořit jej takovým způsobem, aby jej šlo pohodlně programovat, rozšiřovat a používat.

Pokud se Vám program líbil, pracujte na něm dále. Vytvořte třídu Škola a v ní úložiště pro pedagogy. Přidejte možnost vytvořit více školních tříd. Přidejte ke třídě písmeno. Vytvořte grafické uživatelské rozhraní.. je toho tolik, v čem lze program rozvíjet a zdokonalovat. Vždy však bude pohodlnější a přehlednější, když k práci využijete výhod objektově orientovaného programování.